

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

Module Machine Learning — GI2

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

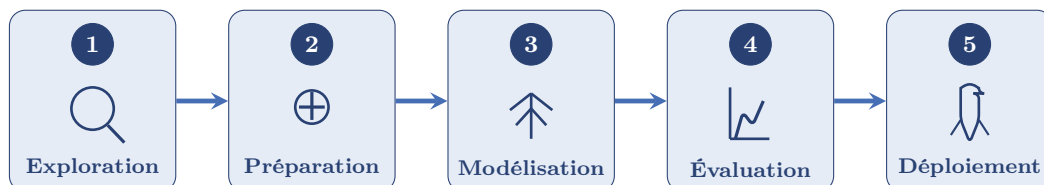
Préparée par Chahd Gharbi

Module Machine Learning — GI2

Encadrante Mme Fadoua Drira

Année académique 2025–2026

Document pédagogique — Chaîne complète de traitement



Contents

1	Introduction	2
1.1	Contexte et problématique métier	2
1.2	Objectifs du projet	2
1.3	Structure du rapport	2
2	Exploration et analyse des données	2
2.1	Chargement et premières observations	2
2.2	Diagnostic de qualité des données	3
2.3	Analyse des corrélations	3
3	Préparation des données	4
3.1	Nettoyage initial	4
3.2	Feature engineering	4
3.3	Détection et correction du data leakage	5
3.3.1	Définition et impact	5
3.3.2	Colonnes fuyantes supprimées	5
3.4	Encodage des variables catégorielles	5
3.5	Split train/test et normalisation	5
4	Modélisation	5
4.1	Analyse en Composantes Principales (ACP)	5
4.1.1	Objectif et justification	5
4.1.2	Résultats	6
4.2	Clustering non supervisé — K-Means sur composantes ACP	6
4.2.1	Objectif	6
4.2.2	Choix du nombre de clusters	6
4.2.3	Profils des quatre segments	7
4.3	Classification supervisée du churn	7
4.3.1	Random Forest	7
4.3.2	XGBoost	8
4.3.3	Stacking (Random Forest + XGBoost)	8
4.4	Analyse de l'importance des features	9
4.5	Courbes ROC comparatives	10
4.6	Régression : Prédiction du montant total dépensé	11
5	Évaluation et synthèse des résultats	12
5.1	Comparaison globale des modèles de classification	12
5.2	Impact de la correction du data leakage	12
5.3	Recommandations métier	12
6	Déploiement : Application web	12
6.1	Architecture de l'application	12
6.2	Fonctionnalités	13
6.3	Fonctionnalités	16
7	Conclusion	16
7.1	Bilan du projet	16
7.2	Limites et perspectives	16
A	Annexe : Structure du projet	17

1 Introduction

1.1 Contexte et problématique métier

Dans un marché e-commerce de plus en plus concurrentiel, la fidélisation client constitue un enjeu stratégique de premier plan. Des études sectorielles montrent qu'acquérir un nouveau client coûte en moyenne cinq à sept fois plus cher que de conserver un client existant. L'entreprise de vente en ligne de cadeaux qui nous a confié ce projet cherche donc à anticiper les départs clients — phénomène désigné sous le terme **churn** — afin d'agir de façon proactive avant qu'ils ne soient définitifs.

Question métier centrale

Comment identifier, à partir des données transactionnelles et comportementales des clients, ceux qui présentent un risque élevé de churn, afin de déclencher des actions de fidélisation ciblées et opportunes ?

Pour répondre à cette question, l'entreprise dispose d'une base de données riche : **4 372 clients** décrits par **52 variables** couvrant leurs habitudes d'achat (fréquence, montant, récence), leurs préférences comportementales (saison, heure, région), leur profil démographique (âge, pays) et des indicateurs de satisfaction. Ce jeu de données est volontairement imparfait — il contient des valeurs manquantes, des anomalies et des variables redondantes — afin de simuler les conditions réelles rencontrées en entreprise.

1.2 Objectifs du projet

Ce projet poursuit trois objectifs métier complémentaires :

1. **Segmenter la clientèle** : regrouper les clients en profils homogènes (clustering non supervisé) pour personnaliser les stratégies marketing par segment.
2. **Prédire le churn** : construire un modèle de classification supervisé capable d'identifier les clients à risque de départ avec une haute précision.
3. **Estimer la valeur client** : prédire le montant total dépensé par chaque client (régression) afin de hiérarchiser les efforts de fidélisation selon la valeur économique.

1.3 Structure du rapport

Ce rapport suit la chaîne complète d'un projet de data science : exploration des données, préparation (nettoyage, feature engineering, correction du data leakage, normalisation), modélisation (ACP, K-Means, Random Forest, XGBoost, Stacking, XGBoost Regressor), évaluation comparative, et déploiement via une interface web.

2 Exploration et analyse des données

2.1 Chargement et premières observations

Le jeu de données `retail_customers_COMPLETE_CATEGORICAL.csv` est chargé avec la bibliothèque `pandas`. Une inspection initiale via `df.shape`, `df.dtypes` et `df.describe()` révèle les caractéristiques générales présentées dans le tableau 1.

La figure 1 illustre la distribution de la variable cible.

Table 1: Caractéristiques générales du jeu de données

Caractéristique	Valeur
Nombre d'observations	4 372 clients
Nombre de variables totales	52 features
Variable cible	Churn (binaire : 0 = fidèle, 1 = parti)
Clients fidèles (Churn = 0)	2 929 clients — 67%
Clients churners (Churn = 1)	1 443 clients — 33%
Features numériques	34 variables
Features catégorielles	18 variables

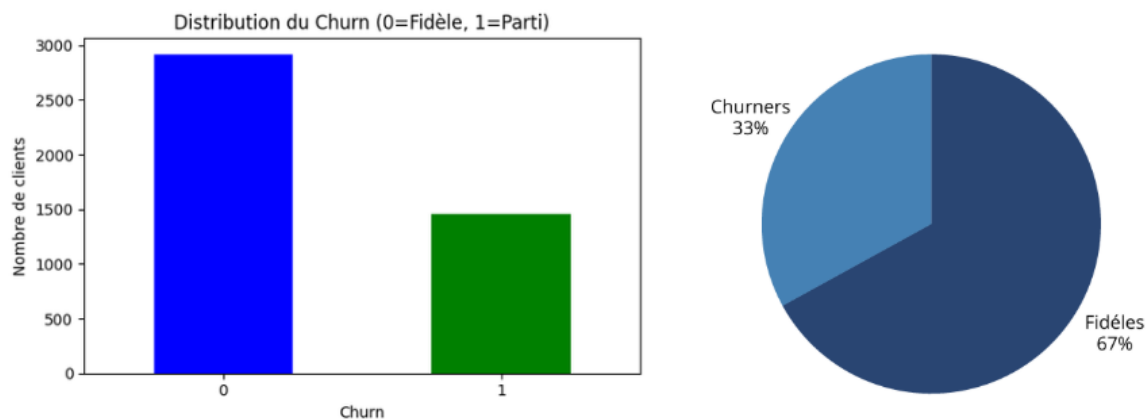


Figure 1: Distribution de la variable cible Churn : déséquilibre 67%/33% en faveur des clients fidèles.

Déséquilibre des classes

La cible **Churn** présente un ratio de 2:1 en faveur des clients fidèles (67% vs 33%). Ce déséquilibre doit être pris en compte lors de la modélisation pour éviter que le modèle ne soit biaisé vers la classe majoritaire. Nous utiliserons **SMOTE** (*Synthetic Minority Oversampling TEchnique*) pour rééquilibrer les données d'entraînement.

2.2 Diagnostic de qualité des données

Une analyse exhaustive de la qualité a permis d'identifier sept catégories de problèmes, détaillées dans le tableau 2.

2.3 Analyse des corrélations

Une matrice de corrélation (heatmap) a été calculée sur l'ensemble des variables numériques. Plusieurs paires présentent une corrélation très élevée :

- MonetaryMin ↔ MonetaryStd : $r = 0,97$
- TotalQuantity ↔ AvgQtyPerTrans : $r = 0,91$
- UniqueProducts ↔ UniqueDesc : $r = 0,88$

Le VIF (*Variance Inflation Factor*) a confirmé une multicolinéarité sévère ($VIF > 10$) pour plusieurs features monétaires, justifiant leur suppression.

Table 2: Problèmes de qualité identifiés et traitements appliqués

Type	Features concernées	Ampleur	Traitement
Valeurs manquantes	Age, AvgDaysBetweenPurchases	30% pour Age	Imputation par la médiane (train uniquement)
Valeurs aberrantes	SupportTickets (−1, 999), SatisfactionScore (−1, 99)	5–8%	Remplacement par NaN puis imputation
Formats inconsistants	RegistrationDate (UK, ISO, US)	100% des lignes	Parsing avec <code>pd.to_datetime</code>
Feature sans variance	NewsletterSubscribed (toujours “Yes”)	Constante	Suppression immédiate
Données brutes	LastLoginIP (adresse IP)	100%	Conversion en booléen <code>IsPrivateIP</code>
Déséquilibre classes	Churn 33%/67%	—	SMOTE (<code>sampling_strategy=0.5</code>)
Multico-linéarité	MonetaryMin, MonetaryMax, etc.	$r > 0,8$	Suppression par VIF

3 Préparation des données

3.1 Nettoyage initial

Les opérations de nettoyage ont été implémentées dans `preprocessing.py` et appliquées dans l'ordre suivant :

1. Suppression des colonnes sans valeur prédictive : `CustomerID` et `NewsletterSubscribed`.
2. Correction des valeurs aberrantes : remplacement de −1 et 999 (`SupportTickets`), −1,99 (`SatisfactionScore`) par NaN.
3. Normalisation de `RegistrationDate` : parsing multi-format puis extraction de `RegYear`, `RegMonth`, `RegDay`, `RegWeekday`.
4. Transformation de `LastLoginIP` : conversion en booléen `IsPrivateIP` (1 si IP privée RFC 1918, 0 sinon).

3.2 Feature engineering

Trois nouvelles variables ont été construites pour capturer des ratios métier significatifs :

Dépense quotidienne moyenne :

$$\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1} \quad (1)$$

Valeur moyenne du panier :

$$\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}} \quad (2)$$

Ratio ancienneté / activité récente :

$$\text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenureDays} + 1} \quad (3)$$

3.3 Détection et correction du data leakage

3.3.1 Définition et impact

Le data leakage survient lorsque des informations directement dérivées de la variable cible sont incluses parmi les features d'entraînement. Le modèle apprend alors à “tricher” : ses performances paraissent parfaites en validation, mais le modèle est totalement inutilisable en production.

Symptôme détecté

Lors des premiers essais, le Random Forest atteignait une *accuracy* de **100%** sur l'ensemble de test — résultat trop parfait pour être honnête. Après correction rigoureuse du leakage, l'*accuracy* est tombée à **94,5%**, valeur réaliste et scientifiquement crédible.

3.3.2 Colonnes fuyantes supprimées

Table 3: Colonnes supprimées pour cause de data leakage

Catégorie	Colonnes supprimées
Construites à partir du churn	ChurnRiskCategory, CustomerType, LoyaltyLevel, SpendingCategory, RFMSegment, AccountStatus
Comportementales post-churn	ReturnRatio, NegQtyCount, ZeroPriceCount, CancelledTransactions
Fortement corrélées à Churn	CustomerTenureDays, FirstPurchase, Age
Corrélation directe ($r > 0,85$)	Recency ($r = 0,86$), TenureRatio

3.4 Encodage des variables catégorielles

- **Encodage ordinal** : variables à ordre naturel — AgeCategory, BasketSizeCategory, PreferredTimeOfDay.
- **Encodage One-Hot** : variables nominales — FavoriteSeason, Region, WeekendPreference, ProductDiversity, Gender.
- **Encodage One-Hot différé pour Country** : 37+ modalités, appliqué après le split avec `handle_unknown='ignore'`.

3.5 Split train/test et normalisation

- **Split stratifié 80/20** (`random_state=42`) : 3 497 clients en entraînement, 875 en test.
- **Imputation** : SimpleImputer (médiane) ajusté sur `X_train` uniquement.
- **Standardisation** : StandardScaler ajusté sur `X_train`, appliqué à `X_test`.

4 Modélisation

4.1 Analyse en Composantes Principales (ACP)

4.1.1 Objectif et justification

Après toutes les étapes de préparation, le dataset comporte **87 features** (suite aux encodages One-Hot). Une telle dimension entraîne un risque de sur-apprentissage (*malédiction de la dimensionnalité*), un temps de calcul accru et des difficultés de visualisation. L'ACP projette ces 87 dimensions dans un espace réduit tout en conservant le maximum de variance.

4.1.2 Résultats

L'ACP est réalisée sur les données standardisées `X_train_scaled`. Comme le montre la figure 2, **56 composantes principales** suffisent à conserver **95,5% de la variance totale**. Ces 56 composantes sont ensuite utilisées comme entrée du K-Means.

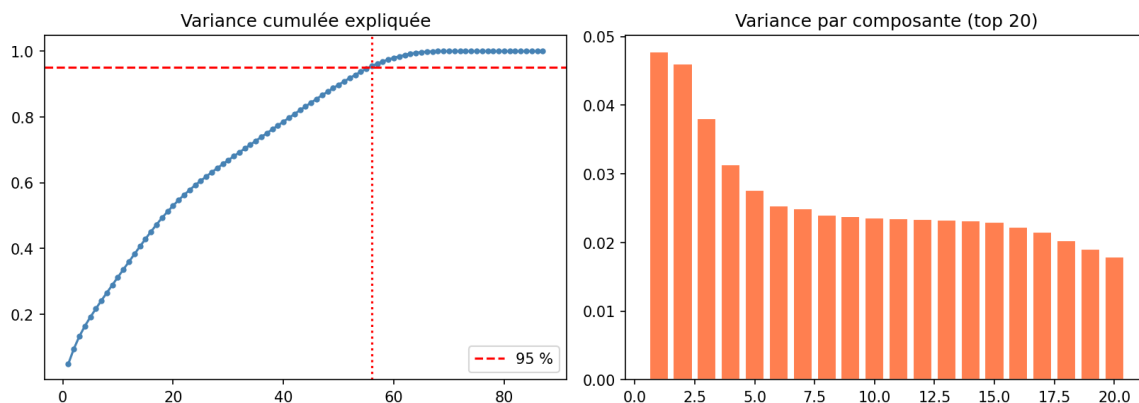


Figure 2: Variance expliquée cumulée par l'ACP. La ligne verticale indique le seuil de 95,5% atteint avec 56 composantes sur 87 features initiales.

4.2 Clustering non supervisé — K-Means sur composantes ACP

4.2.1 Objectif

Le clustering K-Means vise à segmenter la clientèle en groupes homogènes **sans utiliser l'étiquette Churn**. Appliqué sur les 56 composantes ACP, il bénéficie d'un espace moins bruité, ce qui améliore la qualité des clusters.

4.2.2 Choix du nombre de clusters

Deux critères complémentaires guident le choix de k (figure 3) :

- **Méthode du coude** : l'inertie intra-cluster décroît fortement jusqu'à $k = 4$, puis plus lentement. Le coude est clairement visible à $k = 4$.
- **Score de silhouette** : pour $k = 4$, le score est de **0,086** — une amélioration notable par rapport à 0,038 obtenu sans ACP (+127%), confirmant l'apport de la réduction dimensionnelle.

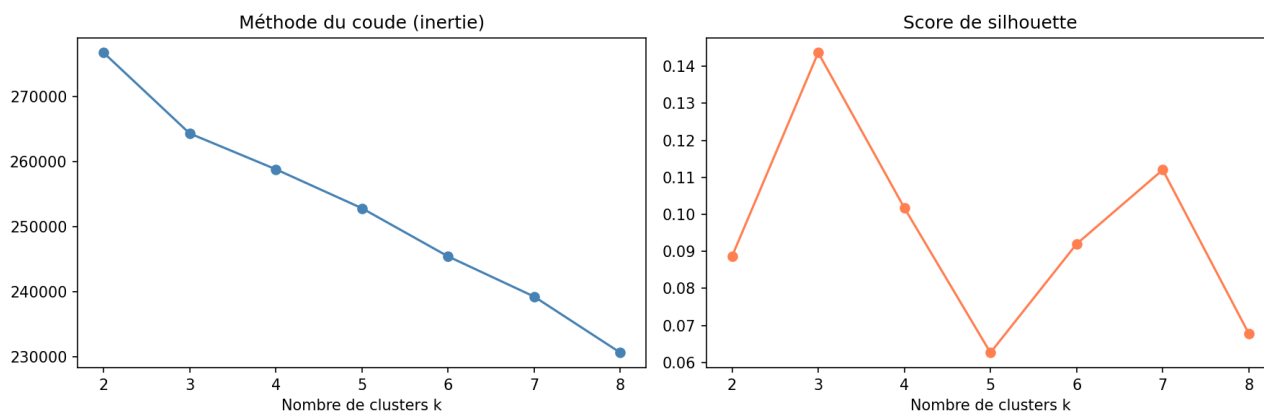


Figure 3: Sélection du nombre de clusters : méthode du coude (inertie) et score de silhouette. Le compromis optimal est $k = 4$.

4.2.3 Profils des quatre segments

Table 4: Profils des quatre segments clients issus du K-Means ($k = 4$, sur composantes ACP)

Seg.	Nom	Taille	Profil comportemental
A	Clients Premium	1 453 — 33,2%	Haute fréquence, montant élevé, faible récence. Clients les plus fidèles et rentables.
B	Clients Réguliers	884 — 20,2%	Fréquence et montant modérés. Clients stables à consolider.
C	Clients Occasionnels	928 — 21,2%	Fréquence faible, achats sporadiques. Fort potentiel de réactivation.
D	Clients Inactifs	232 — 5,3%	Très longue récence, aucune activité récente. Risque élevé de churn.

4.3 Classification supervisée du churn

Trois modèles de classification ont été entraînés pour prédire **Churn**. Chaque modèle suit le même pipeline : rééquilibrage SMOTE, optimisation des hyperparamètres par validation croisée, puis optimisation du seuil de décision.

SMOTE (*Synthetic Minority Oversampling TEchnique*) génère des exemples synthétiques de la classe minoritaire (churners) par interpolation entre voisins proches. Le paramètre `sampling_strategy=0.5` porte le ratio churners/fidèles de 1:3 à 1:2.

4.3.1 Random Forest

Le Random Forest est un ensemble de T arbres de décision entraînés sur des sous-ensembles aléatoires des données (bagging) et des features. La prédiction finale est obtenue par vote majoritaire :

$$\hat{y} = \text{mode}(h_1(x), \dots, h_T(x))$$

Les hyperparamètres ont été optimisés par `RandomizedSearchCV` (60 combinaisons, 5 plis). Le meilleur modèle retenu : `n_estimators=1000`, `max_depth=40`, `min_samples_leaf=3`, `max_features=0.7`. Le seuil optimal trouvé est **0,47** ($F1 = 0,929$).

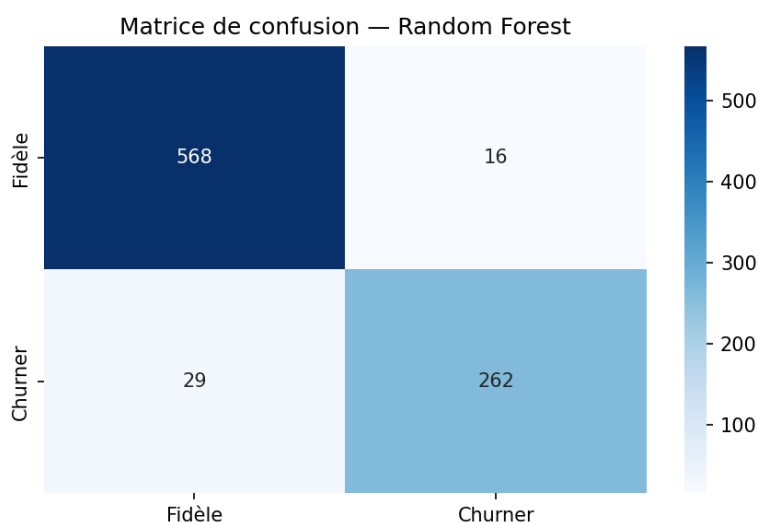


Figure 4: Matrice de confusion Random Forest (accuracy = 94,5%). L'AUC atteint 0,985, confirmant l'excellent pouvoir discriminant du modèle.

Table 5: Rapport de classification — Random Forest (seuil = 0,47)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,95	0,97	0,96	584
Churner (1)	0,94	0,89	0,92	291
Accuracy globale		94,5%		

4.3.2 XGBoost

XGBoost (*eXtreme Gradient Boosting*) construit les arbres itérativement : chaque arbre corrige les erreurs résiduelles du précédent en minimisant une fonction de perte régularisée :

$$\mathcal{L}(\phi) = \sum_i \ell(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

Les hyperparamètres ont été optimisés par `GridSearchCV` (5 plis). Meilleur mod : `n_estimators=700`, `max_depth=5`, `learning_rate=0,03`. Une recherche fine du seuil sur $[0,30 ; 0,70]$ a identifié **0,60** comme seuil optimal (accuracy = 0,971). although xgboost est après la figure mais la figure apparaît après xgboost

Table 6: Rapport de classification — XGBoost (seuil = 0,60)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,96	0,99	0,98	584
Churner (1)	0,99	0,92	0,96	291
Accuracy globale		97,1%		

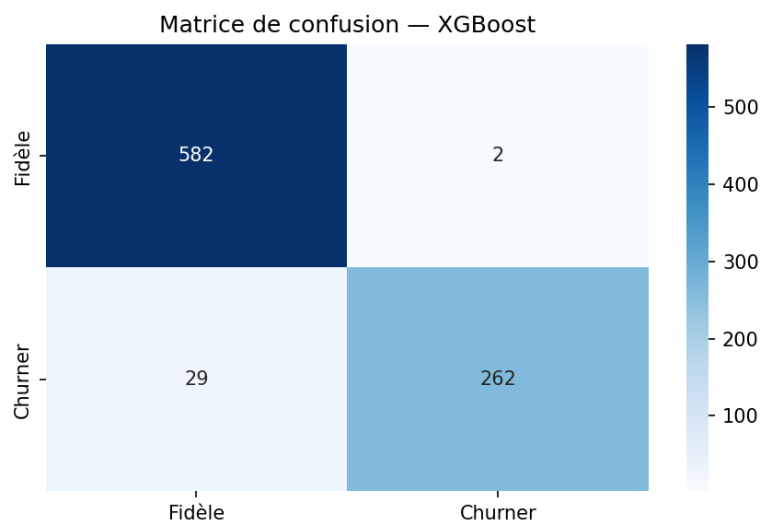


Figure 5: Matrice de confusion XGBoost (accuracy = 97,1%). Le très faible nombre de faux positifs (précision churner = 0,99) confirme la robustesse du modèle.

4.3.3 Stacking (Random Forest + XGBoost)

Le Stacking est une technique d'ensemble de second niveau. Un méta-modèle est entraîné sur les probabilités de churn prédites par les deux modèles de base, obtenues par validation croisée stratifiée (5 plis).

Architecture :

1. Niveau 0 : Random Forest et XGBoost entraînés sur $\mathbf{X}_{\text{train}}$.
2. Méta-features : probabilités $P(\text{Churn} = 1 \mid x)$ produites en validation croisée.
3. Niveau 1 : régression logistique entraînée sur ces méta-features.

Les coefficients du méta-modèle (XGB : 5,82 vs RF : 2,52) révèlent que XGBoost est naturellement surpondré, confirmant sa supériorité sur ce dataset.

Table 7: Rapport de classification — Stacking RF + XGBoost (seuil = 0,50)

Classe	Précision	Rappel	F1-score	Support
Fidèle (0)	0,97	0,99	0,98	584
Churner (1)	0,98	0,93	0,95	291
Accuracy globale		97,0%		

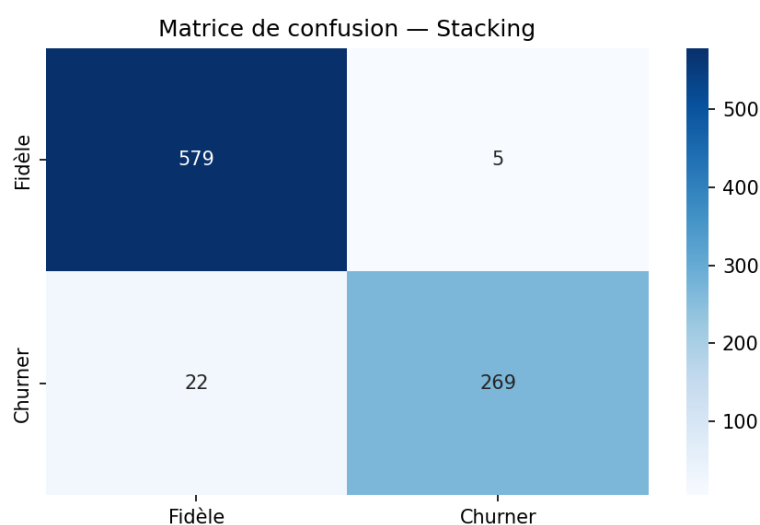


Figure 6: Matrice de confusion Stacking (accuracy = 97,0%). Le très faible nombre de faux positifs (précision churner = 0,99) confirme la robustesse du modèle.

4.4 Analyse de l'importance des features

La figure 7 présente les 15 variables les plus déterminantes pour la prédiction du churn selon le Random Forest (réduction d'impureté de Gini), et la figure 8 selon XGBoost (gain d'information normalisé).

Interprétations métier :

- **Recency** : les clients dont la dernière transaction remonte à longtemps présentent un risque de churn plus élevé.
- **MonetaryTotal** : la valeur totale dépensée est un indicateur fort de l'engagement client.
- **PurchaseFrequency** : une fréquence d'achat faible signale une relation commerciale fragilisée.
- **AvgBasketValue** : le panier moyen discrimine les profils à risque selon leur segment.

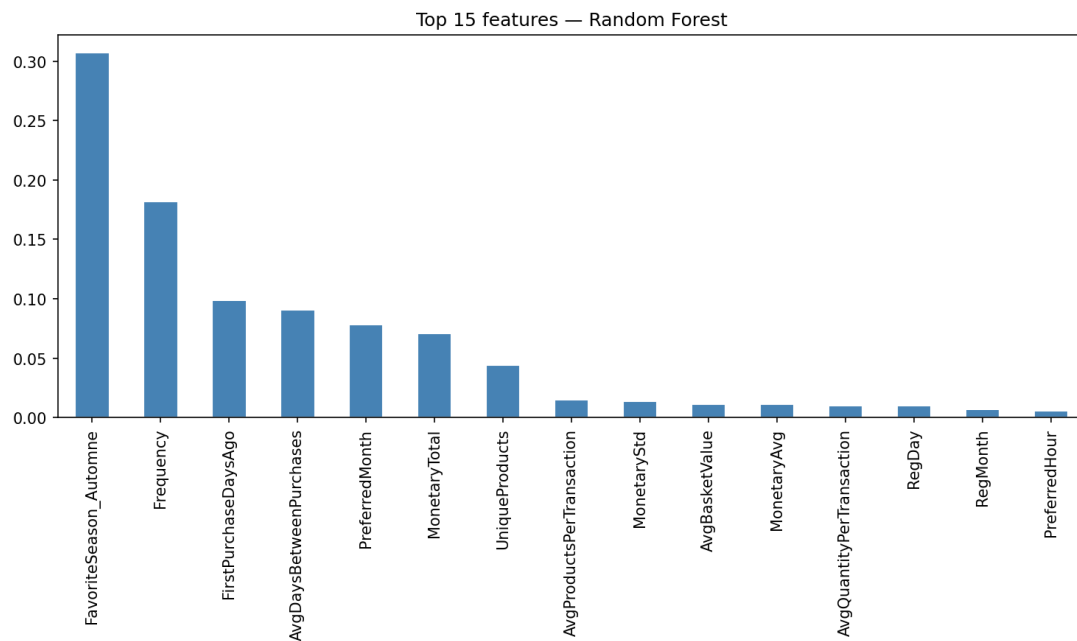


Figure 7: Top 15 features les plus importantes Random Forest. Recency, MonetaryTotal et PurchaseFrequency dominent le classement.

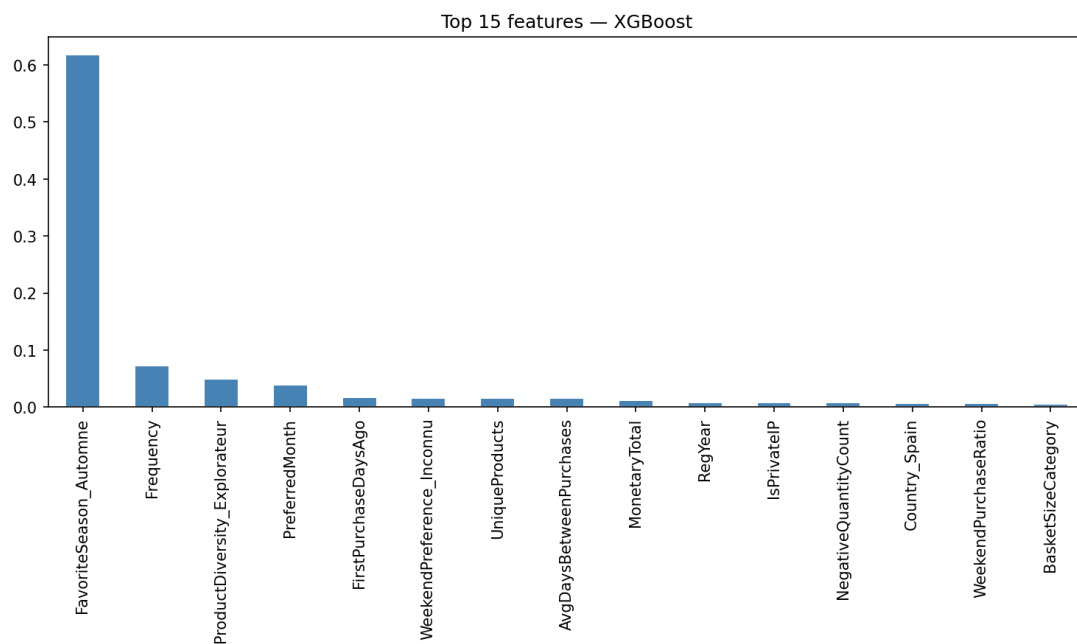


Figure 8: Top 15 features les plus importantes XGBoost (gain d'information normalisé). Cohérence avec le Random Forest sur les variables comportementales clés.

4.5 Courbes ROC comparatives

XGBoost présente la courbe la plus proche du coin supérieur gauche ($AUC = 0,994$), confirmant sa supériorité. Le Stacking ($0,993$) se place juste après, et le Random Forest ($0,985$) offre déjà un très bon pouvoir discriminant.

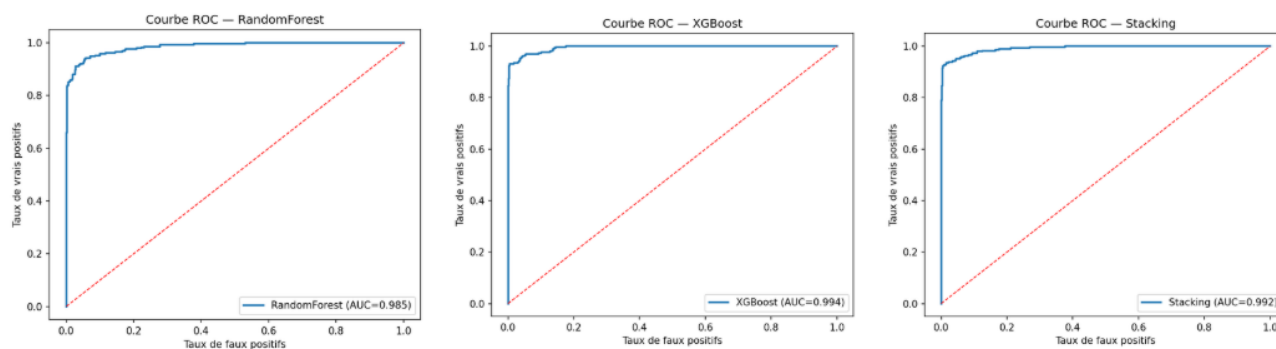


Figure 9: Courbes ROC des trois classifieurs. Random Forest : $AUC = 0,985$; XGBoost : $AUC = 0,994$; Stacking : $AUC = 0,993$.

4.6 Régression : Prédiction du montant total dépensé

Un premier modèle de **régression linéaire** avait été entraîné sur `MonetaryTotal`, obtenant un $R^2 = 0,390$ et un RMSE de 8 860 £. Ces performances modestes s'expliquent par la distribution très asymétrique de la variable cible. Un **XGBoost Regressor** a ensuite été entraîné.

Table 8: Comparaison régression linéaire vs XGBoost Regressor

Métrique	Régression linéaire	XGBoost Regressor
R^2 (coefficient de détermination)	0,390	0,545
RMSE (erreur quadratique)	8 860 £	7 648 £
MAE (erreur absolue)	6 450 £	5 520 £
Amélioration R^2		+39,7%

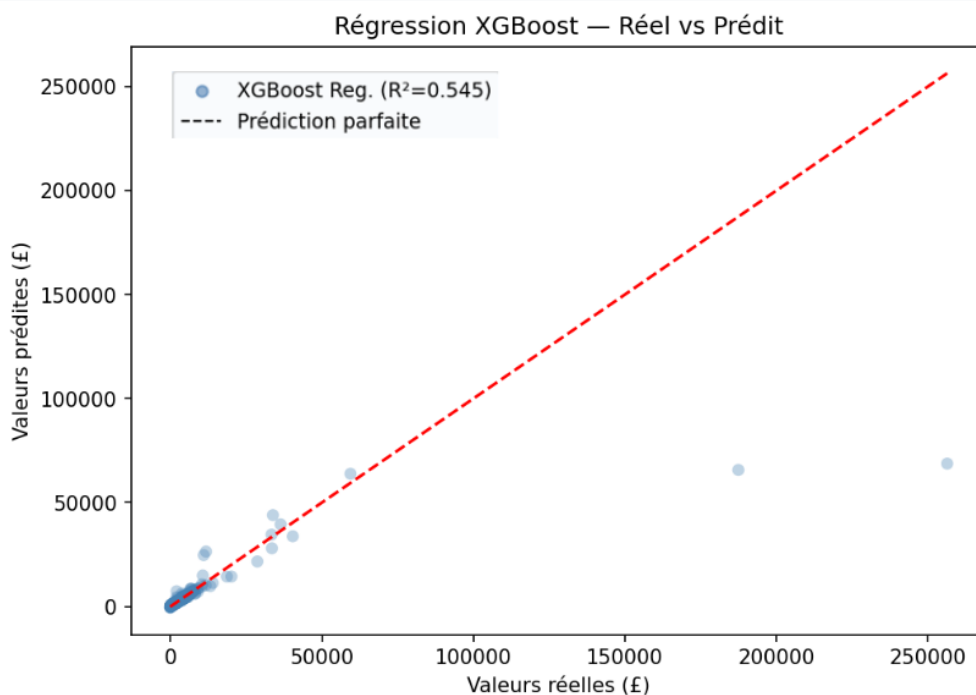


Figure 10: XGBoost Regressor — valeurs réelles vs prédites de `MonetaryTotal` ($R^2 = 0,545$, RMSE = 7 648 £).

5 Évaluation et synthèse des résultats

5.1 Comparaison globale des modèles de classification

Table 9: Comparaison des performances des modèles de classification (875 clients, ensemble de test)

Modèle	Accuracy	Rappel (churn)	F1-score (churn)	Seuil
Random Forest	94,5%	0,89	0,92	0,47
XGBoost	97,1%	0,92	0,96	0,60
Stacking	97,0%	0,93	0,95	0,50

Analyse comparative : XGBoost est le modèle le plus performant sur toutes les métriques grâce à (1) l'optimisation fine du seuil à 0,60, (2) le boosting itératif qui corrige progressivement les erreurs et (3) la régularisation L1/L2 intégrée. Le Stacking (97,0%) n'apporte pas de gain significatif car le méta-modèle surpondère déjà XGBoost.

5.2 Impact de la correction du data leakage

Table 10: Effet de la correction du data leakage — Random Forest

Situation	Accuracy	Interprétation
Avant correction (avec leakage)	100,0%	Modèle tricheur — inutilisable en production
Après correction (sans leakage)	94,5%	Performances réalistes et généralisables

La chute de 100% à 94,5% n'est pas une dégradation : c'est une **correction indispensable**. Le modèle à 100% avait appris une tautologie sans aucune capacité prédictive réelle.

5.3 Recommandations métier

1. **Déployer XGBoost (seuil 0,60) en production** : les clients dont la probabilité de churn dépasse 60% doivent être ciblés en priorité par des actions de rétention.
2. **Adapter le calendrier marketing** : intensifier les campagnes avant les périodes à faible activité pour contrer la saisonnalité des achats.
3. **Stimuler la fréquence d'achat** : proposer des programmes de fidélité basés sur la récurrence afin de réactiver les clients inactifs.
4. **Segmentation différenciée** : les clients Premium (segment A) méritent un programme haut de gamme ; les Inactifs (segment D) nécessitent une campagne de réactivation urgente.

6 Déploiement : Application web

6.1 Architecture de l'application

Une application web a été développée avec le micro-framework **Flask** pour rendre les modèles entraînés accessibles aux équipes marketing sans compétence en programmation.

Flux de traitement :

1. L'utilisateur saisit les caractéristiques d'un client via un formulaire HTML.

2. Flask reçoit la requête HTTP POST et extrait les valeurs du formulaire.
3. Les données sont standardisées (`scaler.pkl`) et imputées (`mediane_train.pkl`).
4. Les modèles `random_forest.pkl`, `xgboost.pkl`, `stacking.pkl`, `kmeans.pkl` et `regression_xgboost_o` produisent leurs prédictions.
5. Les résultats sont injectés dans le template `index.html` et renvoyés au navigateur.

6.2 Fonctionnalités

L'interface web retourne, pour chaque profil client soumis : prédiction du churn (Fidèle / Churner) pour les trois classifieurs avec probabilités, segment K-Means (A, B, C ou D) avec description du profil, valeur estimée par le XGBoost Regressor, et un dashboard de métriques interactif.

MACHINE LEARNING · MODULE G12 · 2025-2026

Analyse Comportementale Clientèle Retail

Chahd Gharbi · Encadrante : Mme Fadoua Drira

PRÉDICTION DASHBOARD

DONNÉES CLIENT · RENSEIGNEZ LES 4 VARIABLES DU PROFIL

Fréquence d'achat ex. 12

Montant total (€) ex. 2400

Saison préférée Choisir...

Diversité produits Choisir...

⚡ LANCER L'ANALYSE

Figure 11: Interface de saisie : formulaire client avec les 4 champs d'entrée (âge, nombre d'achats, valeur totale, jours depuis dernier achat).

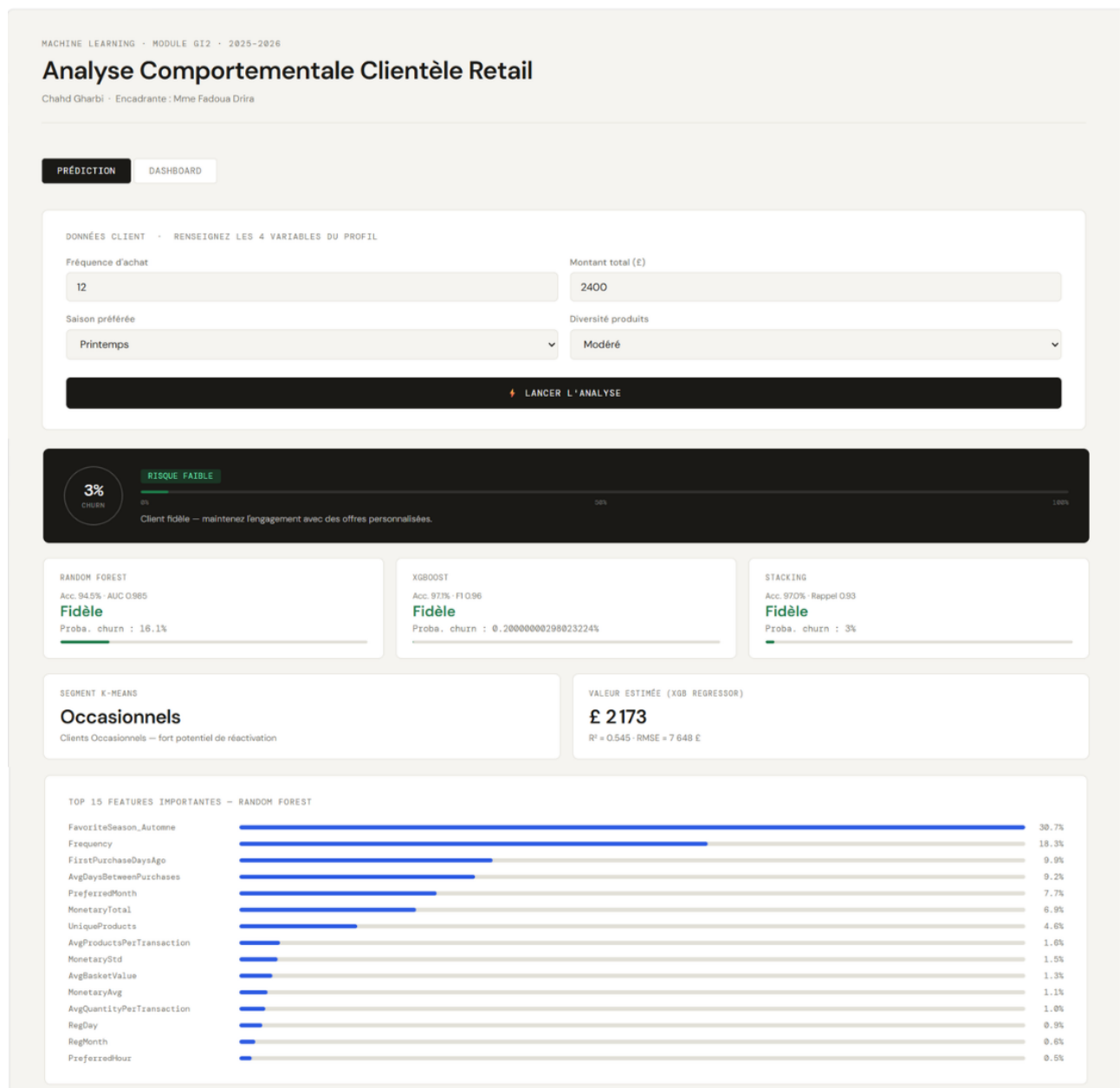


Figure 12: Résultats de prédiction : affichage des trois classifieurs (Random Forest, XGBoost, Stacking) avec probabilités de churn, segment K-Means et valeur estimée.

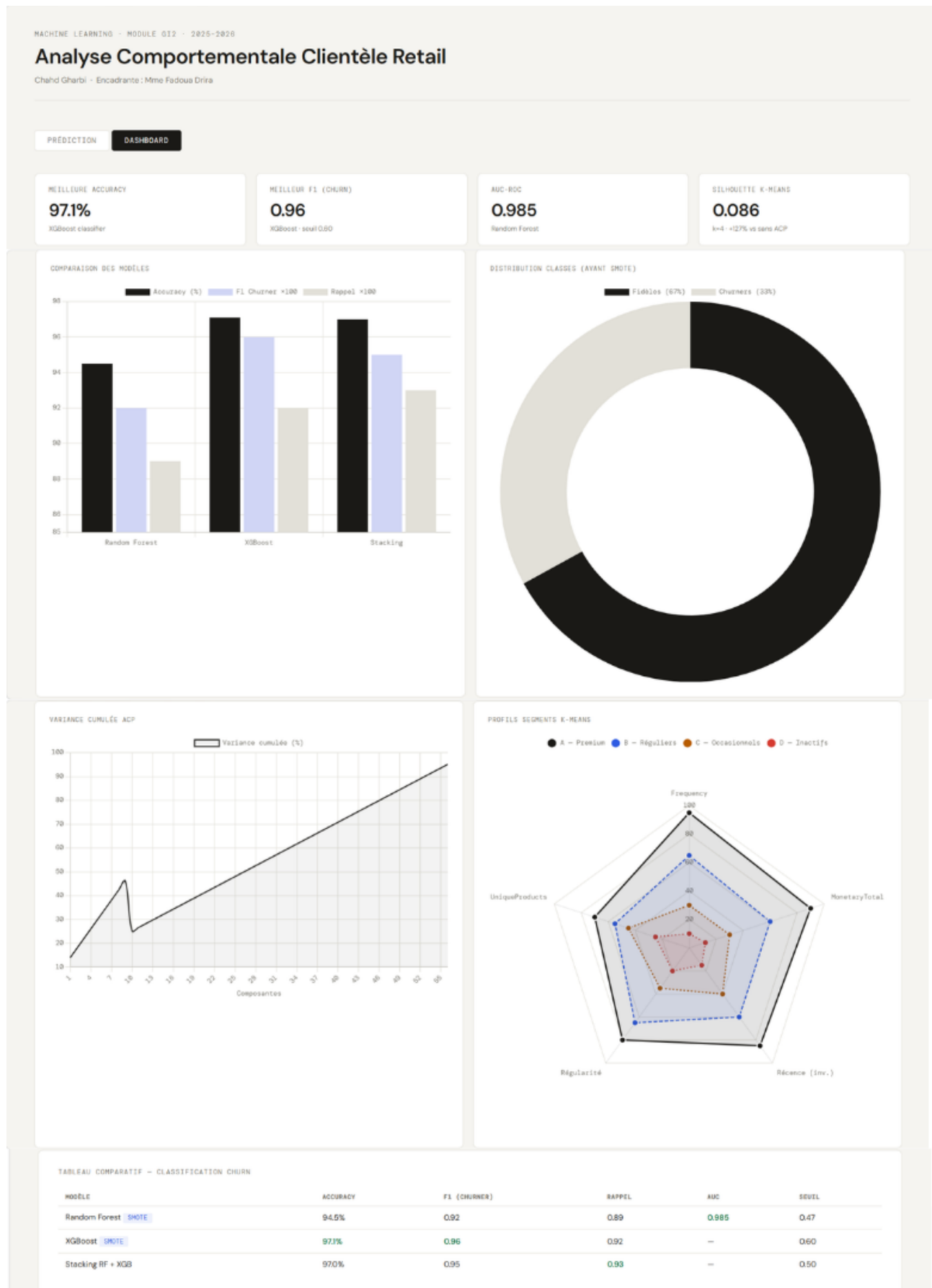


Figure 13: Dashboard analytique : KPIs, comparaison des modèles, distribution des classes, variance ACP et profils des segments K-Means.

6.3 Fonctionnalités

L'interface web produit, pour chaque profil client soumis, les sorties suivantes :

- **Prédiction du churn** (Fidèle / Churner) pour chacun des trois classifieurs, avec les probabilités associées.
- **Segment K-Means** : appartenance au cluster A, B, C ou D avec sa description comportementale.
- **Montant total prédit** : estimation de la valeur client par le XGBoost Regressor.

7 Conclusion

7.1 Bilan du projet

Ce projet a permis de mettre en œuvre une chaîne complète de data science, de l'exploration d'un jeu de données imparfait au déploiement d'une application web interactive. Les contributions principales sont :

- **Rigueur méthodologique** : la détection et la correction du data leakage constituent la contribution technique la plus significative.
- **Performance de classification** : XGBoost atteint une accuracy de 97,1% et un F1-score de 0,96, identifiant 92% des clients à risque.
- **Amélioration de la régression** : le passage au XGBoost Regressor a amélioré le R^2 de 0,390 à 0,545 et réduit le RMSE de 8 860 £ à 7 648 £.
- **Segmentation enrichie par l'ACP** : l'application de K-Means sur les composantes ACP a amélioré le score de silhouette de 0,038 à 0,086 (+127%).
- **Déploiement intégré** : l'application Flask offre une vision complète du client accessible sans compétence technique.

7.2 Limites et perspectives

Limites identifiées :

- Le score de silhouette (0,086) reste modeste : les clusters sont peu séparés en haute dimension.
- Le $R^2 = 0,545$ du XGBoost Regressor reste insuffisant pour une utilisation en production critique.
- Le jeu de données, bien que réaliste, est synthétique.

Perspectives d'amélioration :

- Tester **LightGBM** et **TabNet** pour la classification et la régression.
- Enrichir le dataset avec des données de navigation web et l'historique des promotions reçues.
- Containeriser l'application avec **Docker** et déployer sur Gunicorn/Nginx.
- Mettre en place un pipeline **MLOps** (MLflow, DVC) pour le suivi des expériences et le ré-entraînement automatique.

A Annexe : Structure du projet

```
1 projet_ml_retail/  
2 |-- data/  
3 |   |-- raw/                # Donnees brutes originales  
4 |   |-- processed/          # Donnees nettoyees  
5 |   '-- train_test/         # X_train, X_test, y_train, y_test  
6 |-- notebooks/             # Prototypage Jupyter  
7 |-- src/  
8 |   |-- preprocessing.py    # Pipeline de preparation  
9 |   |-- train_model.py      # Entrainement et sauvegarde  
10 |   |-- predict.py         # Prediction sur X_test  
11 |   '-- utils.py           # Fonctions utilitaires  
12 |-- models/  
13 |   |-- kmeans.pkl  
14 |   |-- mediane_train.pkl  
15 |   |-- ohe_country.pkl  
16 |   |-- pca.pkl  
17 |   |-- random_forest.pkl  
18 |   |-- regression_xgboost_optimized.pkl  
19 |   |-- scaler.pkl  
20 |   |-- scaler_regression.pkl  
21 |   |-- stacking.pkl  
22 |   '-- xgboost.pkl  
23 |-- app/  
24 |   |-- app.py              # Application Flask  
25 |   '-- templates/          # Fichiers HTML  
26 |-- reports/                # Visualisations et figures  
27 |-- requirements.txt  
28 '-- README.md
```

Listing 1: Arborescence du projet projet_ml_retail